

Exercise 5 — API Documentation

Exercise Description

Use AI to create comprehensive API documentation. Apply Prompt 1 (Endpoint Documentation Generation), Prompt 2 (API Reference Conversion → OpenAPI), and Prompt 3 (API Usage Guide Creation). You do **not** need to run the code — work from extracts to produce the documentation.

Endpoint chosen: the provided Python/Flask **User Registration** endpoint — `POST /api/users/register`.

Prompt 1 — Endpoint Documentation

`POST /api/users/register`

Registers a new user account. Validates required fields, checks username/email uniqueness, validates email format and password strength, hashes the password, persists the user, and triggers a confirmation email (best-effort).

Authentication: none (public registration endpoint).

Request body (application/json):

Field	Type	Required	Rules
username	string	yes	must be unique
email	string	yes	must match <code>^[^@]+@[^@]+\.[^@]+\$</code> ; stored lower-cased; must be unique
password	string	yes	minimum 8 characters

Responses:

Code	Meaning	Body
201 Created	User registered	<code>{ "message": "User registered successfully", "user": { id, username, email, created_at, role } } (password never returned)</code>
400 Bad Request	Missing field / invalid email / weak password	<code>{ "error": "...", "message": "..." }</code>
409 Conflict	Username taken or email already exists	<code>{ "error": "Username taken" \ "Email exists", "message": "..." }</code>
500 Internal Server Error	Persistence failure (rolled back)	<code>{ "error": "Server error", "message": "Failed to register user" }</code>

Notes / special considerations: the confirmation email is sent inside a `try/except` and its failure is logged but does **not** fail the request. The password is stored only as a hash (`generate_password_hash`). No rate limiting is present in the sample — a real deployment should add it to this endpoint.

Example requests:

```
# Success
POST /api/users/register
{ "username": "johndoe", "email": "john@example.com", "password": "securepass123" }
→ 201 { "message": "User registered successfully", "user": { "id": 1, "username": "johndoe", ... } }

# Weak password
POST /api/users/register
{ "username": "amy", "email": "amy@example.com", "password": "short" }
→ 400 { "error": "Weak password", "message": "Password must be at least 8 characters long" }

# Duplicate email
→ 409 { "error": "Email exists", "message": "An account with this email already exists" }
```

Prompt 2 — OpenAPI (Swagger) conversion

```
openapi: 3.0.0
info:
  title: User API
  version: 1.0.0
paths:
  /api/users/register:
    post:
      summary: Register a new user
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [username, email, password]
              properties:
                username: { type: string }
                email: { type: string, format: email }
                password: { type: string, minLength: 8, format: password }
      responses:
        '201':
          description: User registered successfully
```

```

    content:
      application/json:
        schema:
          type: object
          properties:
            message: { type: string }
            user:
              type: object
              properties:
                id: { type: integer }
                username: { type: string }
                email: { type: string, format: email }
                created_at: { type: string, format: date-time }
                role: { type: string }
            '400': { description: Missing field, invalid email, or weak password }
            '409': { description: Username or email already exists }
            '500': { description: Server error }
  components:
    schemas:
      Error:
        type: object
        properties:
          error: { type: string }
          message: { type: string }

```

Prompt 3 — Developer Usage Guide

Audience: developers integrating the sign-up flow. **Tone:** friendly but precise.

1. **No auth needed** — this is the entry point that *creates* accounts, so no token is required.
2. **Send JSON** with `Content-Type: application/json` containing `username`, `email`, `password`.
3. **Handle the happy path** — on `201` you receive the created `user` object (no password); persist the `id` if you need it.
4. **Handle validation** — surface `400` messages directly to the user (they are human-readable), and treat `409` as "already registered → offer sign-in instead".
5. **Example (Python requests):**

```

import requests
r = requests.post(
    "https://api.example.com/api/users/register",
    json={"username": "johndoe", "email": "john@example.com", "password":
"securepass123"},
)
if r.status_code == 201:

```

```
user = r.json()["user"]
elif r.status_code in (400, 409):
    show_error(r.json()["message"])
else:
    show_error("Something went wrong, please try again.")
```

Submission for Exercise 5 — API Documentation

- 1. Original API endpoint code:** the provided Flask `POST /api/users/register` (`register_user`) handler.
- 2. Comprehensive endpoint documentation (Prompt 1):** purpose, auth (none), request-body table with validation rules, full response/status-code table, notes on best-effort email + missing rate limiting, and worked example requests.
- 3. Converted format (Prompt 2):** a valid OpenAPI 3.0 document with request/response schemas and status codes.
- 4. Developer usage guide (Prompt 3):** friendly 5-step guide with error-handling advice and a Python `requests` example.
- 5. Notes:** hardest to document were the *implicit* behaviours (email failure not blocking registration; email lower-casing) — only visible by reading the handler, not the signature. Markdown tables were the most effective format for the request/response contract; OpenAPI is best for tooling.